

Semana 10 - Fundamentos de Redes Neurais Convolucionais

João Florindo

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas - Brasil
florindo@unicamp.br

Outline

- 1 Visão Computacional
- 2 Convolução
- 3 Outras Operações Importantes
- 4 Convoluções sobre Volumes
- 5 Uma Camada da Rede Convolucional
- 6 Exemplo de uma Rede Convolucional Simples
- 7 Camadas de *Pooling*
- 8 Exemplo Mais Completo de CNN
- 9 Por que Convoluções?

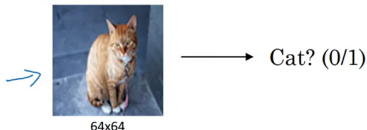
Visão Computacional

- Aplicações como carro autônomo, desbloqueio de um celular ou de uma fechadura usando reconhecimento facial, etc.
- Exemplos de problemas: classificação de imagens, detecção de objetos, transferência de estilo

Visão Computacional

- Aplicações como carro autônomo, desbloqueio de um celular ou de uma fechadura usando reconhecimento facial, etc.
- Exemplos de problemas: classificação de imagens, detecção de objetos, transferência de estilo

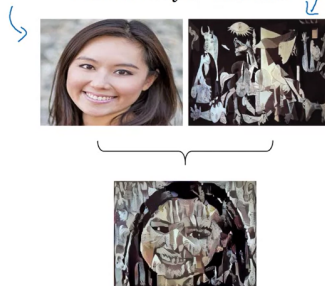
Image Classification



Object detection



Neural Style Transfer



Visão Computacional

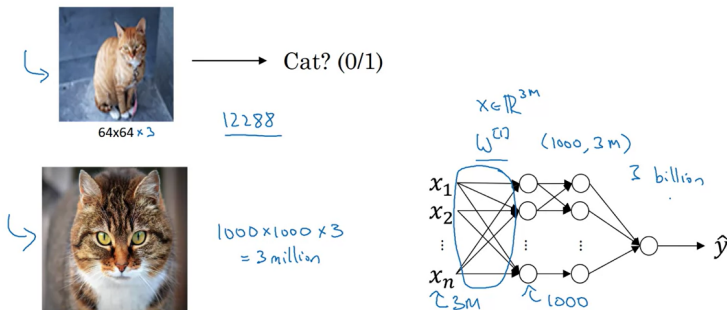
- Problema: tamanho da entrada!
- Imagem 64x64 não é realista. Imagem 1000x1000 e 3 canais de cor (1 megapixel) tem 3 milhões de valores
- Uma rede totalmente conectada com 1000 neurônios escondidos teria 3 bilhões de pesos só na camada escondida!

Visão Computacional

- Problema: tamanho da entrada!
- Imagem 64x64 não é realista. Imagem 1000x1000 e 3 canais de cor (1 megapixel) tem 3 milhões de valores
- Uma rede totalmente conectada com 1000 neurônios escondidos teria 3 bilhões de pesos só na camada escondida!

Visão Computacional

- Problema: tamanho da entrada!
- Imagem 64x64 não é realista. Imagem 1000x1000 e 3 canais de cor (1 megapixel) tem 3 milhões de valores
- Uma rede totalmente conectada com 1000 neurônios escondidos teria 3 bilhões de pesos só na camada escondida!



Visão Computacional

Isso traz dois problemas:

- Custo computacional
- Muitas imagens de treino para evitar *overfitting*

Solução: CONVOLUÇÃO!

Visão Computacional

Isso traz dois problemas:

- Custo computacional
- Muitas imagens de treino para evitar *overfitting*

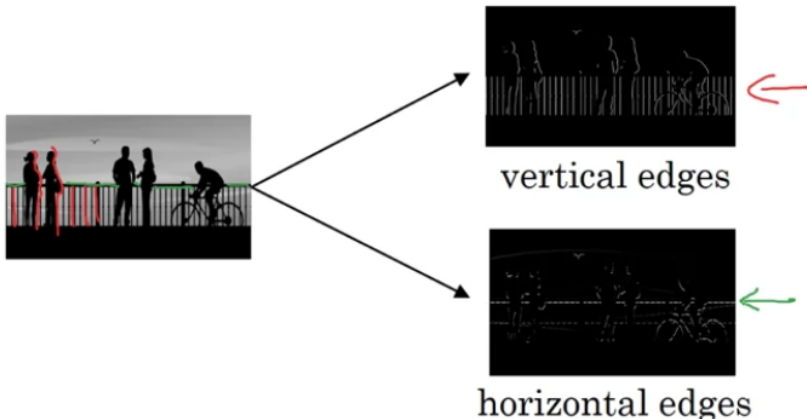
Solução: CONVOLUÇÃO!

Outline

- 1 Visão Computacional
- 2 **Convolução**
- 3 Outras Operações Importantes
- 4 Convoluções sobre Volumes
- 5 Uma Camada da Rede Convolucional
- 6 Exemplo de uma Rede Convolucional Simples
- 7 Camadas de *Pooling*
- 8 Exemplo Mais Completo de CNN
- 9 Por que Convoluções?

Detecção de Bordas

- Vimos que, no reconhecimento facial, as primeiras camadas de uma rede profunda detectam bordas
- Mas como isso pode ser feito?



Bordas Verticais

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

"convolution"

*

1	0	-1
1	0	-1
1	0	-1

3x3
filter

=

4x4

Imagem 6x6 submetida a um processo de **convolução** com uma matriz 3x3, chamada de **filtro** (ou **kernel**)

Bordas Verticais

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

"convolution"
*

1	0	-1
1	0	-1
1	0	-1

3x3
filter

=

-5			

4x4

Bordas Verticais

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

"convolution"
*

1	0	-1
1	0	-1
1	0	-1

3x3
filter

=

-5			

4x4

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

"convolution"
*

1	0	-1
1	0	-1
1	0	-1

3x3
filter

=

-5	-4		

4x4

Bordas Verticais

3	<u>0</u>	<u>1</u>	<u>2</u> ¹	<u>7</u> ⁰	<u>4</u> ⁻¹
1	<u>5</u>	<u>8</u>	<u>9</u> ¹	<u>3</u> ⁰	<u>1</u> ⁻¹
2	<u>7</u>	<u>2</u>	<u>5</u> ¹	<u>1</u> ⁰	<u>3</u> ⁻¹
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

"convolution"

*

1	0	-1
1	0	-1
1	0	-1

3x3
filter

=

<u>-5</u>	<u>-4</u>	0	<u>8</u>

4x4

Bordas Verticais

3	<u>0</u>	<u>1</u>	<u>2</u> ¹	<u>7</u> ⁰	<u>4</u> ⁻¹
1	<u>5</u>	<u>8</u>	<u>9</u> ¹	<u>3</u> ⁰	<u>1</u> ⁻¹
2	<u>7</u>	<u>2</u>	<u>5</u> ¹	<u>1</u> ⁰	<u>3</u> ⁻¹
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

"convolution"
*

1	0	-1
1	0	-1
1	0	-1

3x3
filter

=

<u>-5</u>	<u>-4</u>	0	<u>8</u>

4x4

3	<u>0</u>	<u>1</u>	<u>2</u>	<u>7</u>	<u>4</u>
<u>1</u> ¹	<u>5</u> ⁰	<u>8</u> ⁻¹	<u>9</u>	<u>3</u>	<u>1</u>
<u>2</u> ¹	<u>7</u> ⁰	<u>2</u> ⁻¹	<u>5</u>	<u>1</u>	<u>3</u>
<u>0</u> ¹	<u>1</u> ⁰	<u>3</u> ⁻¹	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

"convolution"
*

1	0	-1
1	0	-1
1	0	-1

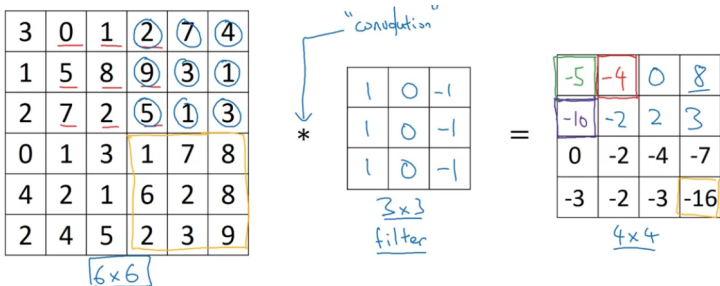
3x3
filter

=

<u>-5</u>	<u>-4</u>	0	<u>8</u>
<u>-10</u>			

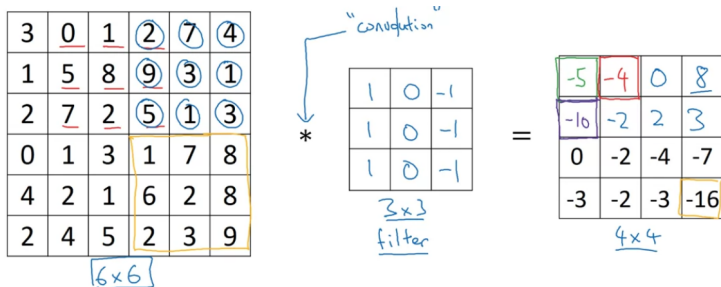
4x4

Bordas Verticais



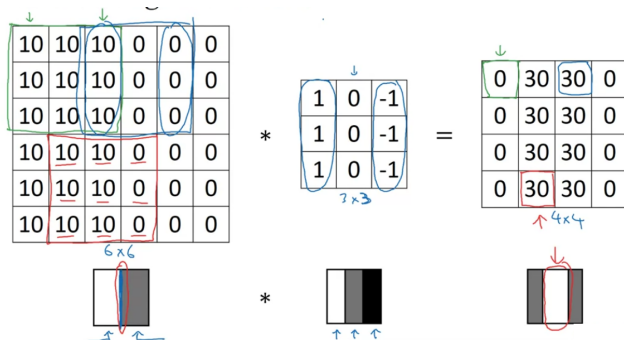
NOTA: Operação implementada em todos os *frameworks* conhecidos de *deep learning*: Python (função *conv-forward*), Tensorflow (*tf.nn.conv2d*), Keras (*Conv2D*)

Bordas Verticais



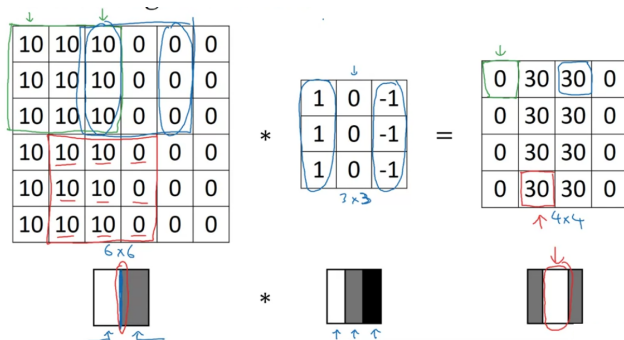
NOTA: Operação implementada em todos os *frameworks* conhecidos de *deep learning*: Python (função *conv-forward*), Tensorflow (*tf.nn.conv2d*), Keras (*Conv2D*)

Exemplo Simplificado



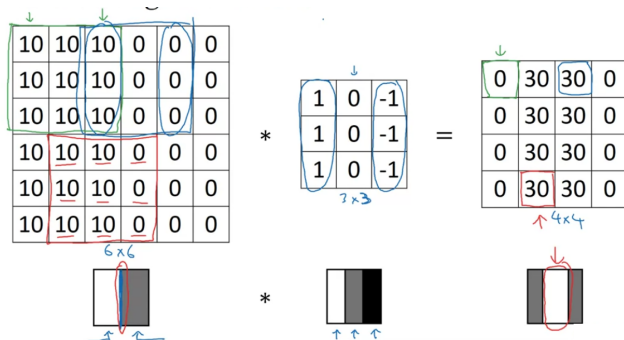
- Borda detectada na região de valores mais altos (branco)
- NOTA: Aqui a borda é "grossa" porque a imagem é muito pequena
- Intuição do filtro de bordas verticais: valores positivos à esquerda, negativos à direita (centro não importa tanto)

Exemplo Simplificado



- Borda detectada na região de valores mais altos (branco)
- NOTA: Aqui a borda é “grossa” porque a imagem é muito pequena
- Intuição do filtro de bordas verticais: valores positivos à esquerda, negativos à direita (centro não importa tanto)

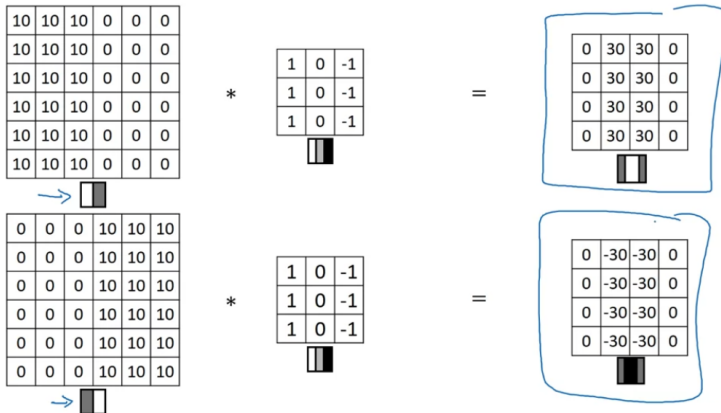
Exemplo Simplificado



- Borda detectada na região de valores mais altos (branco)
- NOTA: Aqui a borda é “grossa” porque a imagem é muito pequena
- Intuição do filtro de bordas verticais: valores positivos à esquerda, negativos à direita (centro não importa tanto)

Ordem de Transição

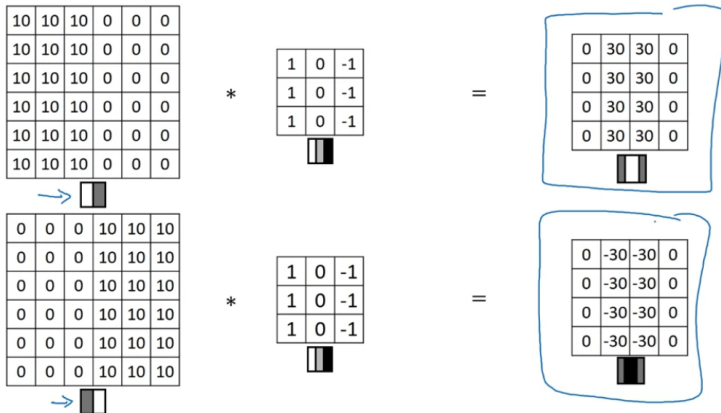
O detector de bordas também identifica a ordem da transição:



NOTA: Se essa ordem não importar, basta tomar os valores absolutos

Ordem de Transição

O detector de bordas também identifica a ordem da transição:



NOTA: Se essa ordem não importar, basta tomar os valores absolutos

Bordas Horizontais



1	0	-1
1	0	-1
1	0	-1

Vertical

1	1	1
0	0	0
-1	-1	-1

Horizontal

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10

6x6

*

1	1	1
0	0	0
-1	-1	-1

=

0	0	0	0
30	10	-10	-30
30	10	-10	-30
0	0	0	0

Transições: no +30 (verde) temos claro em cima e escuro em baixo; no -30, ocorre o contrário; no +/-10 (amarelo), magnitude menor porque há transições nos dois sentidos

Como Escolher os Números no Filtro?

- Grande debate na área
- Alternativas como Sobel e Scharr: peso maior na linha central, aumentando a robustez
- No *deep learning*, estes filtros passam a ser **aprendidos!**

Como Escolher os Números no Filtro?

- Grande debate na área
- Alternativas como Sobel e Scharr: peso maior na linha central, aumentando a robustez

1	0	-1
1	0	-1
1	0	-1



→

1	0	-1
2	0	-2
1	0	-1

Sobel filter



3	0	-3
10	0	-10
3	0	-3

Scharr filter



- No *deep learning*, estes filtros passam a ser **aprendidos**!

Como Escolher os Números no Filtro?

- Grande debate na área
- Alternativas como Sobel e Scharr: peso maior na linha central, aumentando a robustez

1	0	-1
1	0	-1
1	0	-1



→

1	0	-1
2	0	-2
1	0	-1

Sobel filter



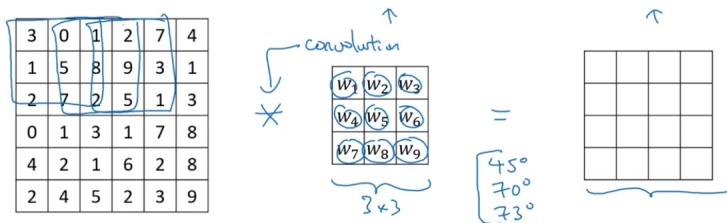
3	0	-3
10	0	-10
3	0	-3

Scharr filter



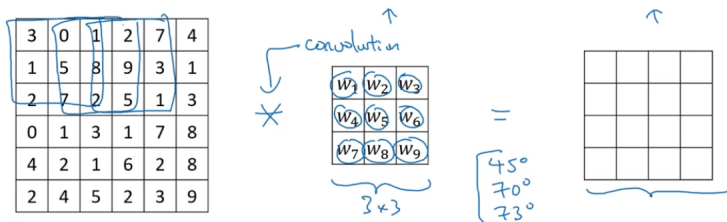
- No *deep learning*, estes filtros passam a ser **aprendidos**!

Como Escolher os Números no Filtro?



- Assim detectamos bordas em direções específicas, p.ex., 45, 70 ou 73°
- Ou outros elementos, sem um nome específico

Como Escolher os Números no Filtro?



- Assim detectamos bordas em direções específicas, p.ex., 45, 70 ou 73°
- Ou outros elementos, sem um nome específico

Outline

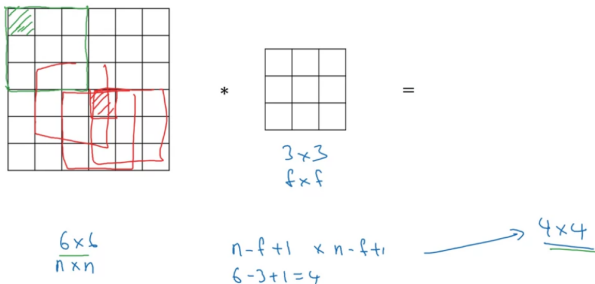
- 1 Visão Computacional
- 2 Convolução
- 3 Outras Operações Importantes**
- 4 Convoluções sobre Volumes
- 5 Uma Camada da Rede Convolutacional
- 6 Exemplo de uma Rede Convolutacional Simples
- 7 Camadas de *Pooling*
- 8 Exemplo Mais Completo de CNN
- 9 Por que Convoluções?

Padding

- A convolução precisa de ajustes para ser usada em uma rede neural
- Vimos que uma imagem 6x6 é reduzida para 4x4
- Isso tem dois efeitos negativos:
 - Imagem vai se tornando cada vez menor quando operada várias vezes como na rede neural
 - Pixel do canto (verde) usado uma vez só na convolução; o do centro (vermelho) é usado muitas vezes

Padding

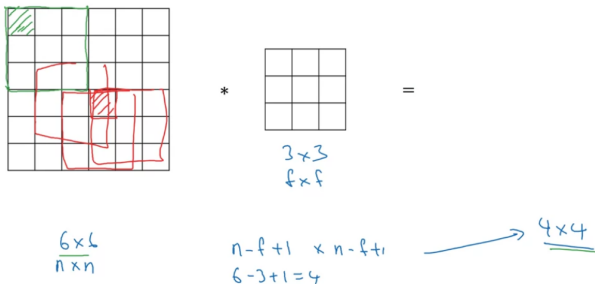
- A convolução precisa de ajustes para ser usada em uma rede neural
- Vimos que uma imagem 6x6 é reduzida para 4x4



- Isso tem dois efeitos negativos:
 - Imagem vai se tornando cada vez menor quando operada várias vezes como na rede neural
 - Pixel do canto (verde) usado uma vez só na convolução; o do centro (vermelho) é usado muitas vezes

Padding

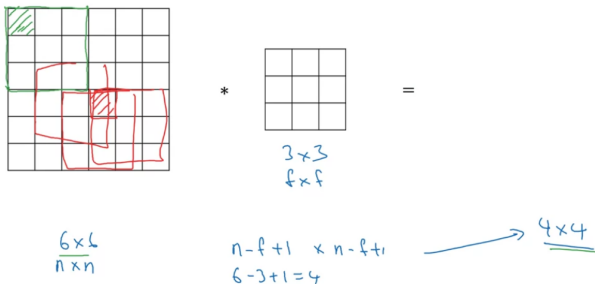
- A convolução precisa de ajustes para ser usada em uma rede neural
- Vimos que uma imagem 6x6 é reduzida para 4x4



- Isso tem dois efeitos negativos:
 - Imagem vai se tornando cada vez menor quando operada várias vezes como na rede neural
 - Pixel do canto (verde) usado uma vez só na convolução; o do centro (vermelho) é usado muitas vezes

Padding

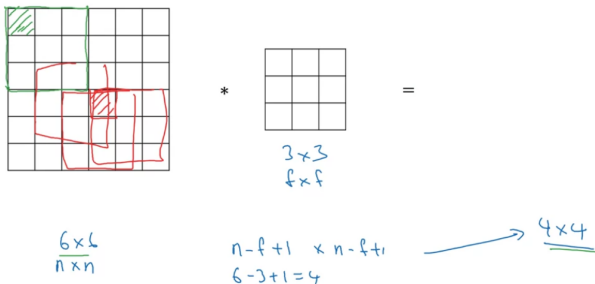
- A convolução precisa de ajustes para ser usada em uma rede neural
- Vimos que uma imagem 6x6 é reduzida para 4x4



- Isso tem dois efeitos negativos:
 - Imagem vai se tornando cada vez menor quando operada várias vezes como na rede neural
 - Pixel do canto (verde) usado uma vez só na convolução; o do centro (vermelho) é usado muitas vezes

Padding

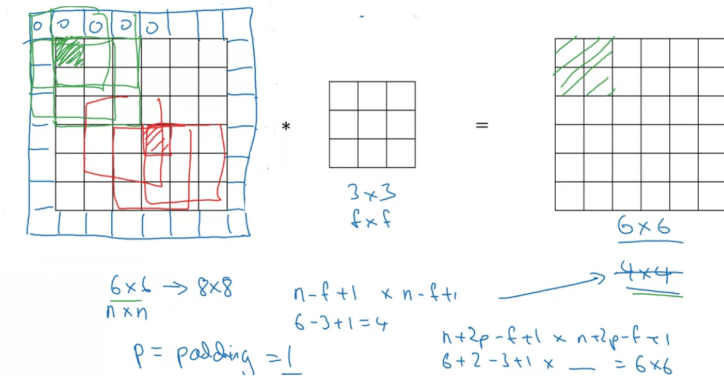
- A convolução precisa de ajustes para ser usada em uma rede neural
- Vimos que uma imagem 6x6 é reduzida para 4x4



- Isso tem dois efeitos negativos:
 - Imagem vai se tornando cada vez menor quando operada várias vezes como na rede neural
 - Pixel do canto (verde) usado uma vez só na convolução; o do centro (vermelho) é usado muitas vezes

Padding

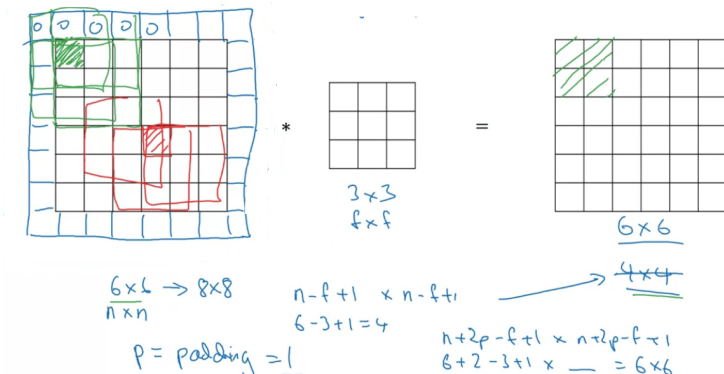
- Resolvido pelo **padding**: adição de uma fileira de pixels em volta



- Note que temos uma nova fórmula para as dimensões da saída
- Por convenção, preenche-se com zeros
- Podemos ter mais fileiras em volta (ex. $p = 2$)

Padding

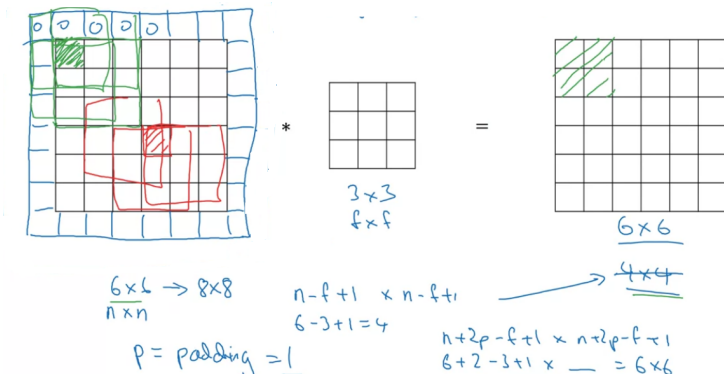
- Resolvido pelo **padding**: adição de uma fileira de pixels em volta



- Note que temos uma nova fórmula para as dimensões da saída
- Por convenção, preenche-se com zeros
- Podemos ter mais fileiras em volta (ex. $p = 2$)

Padding

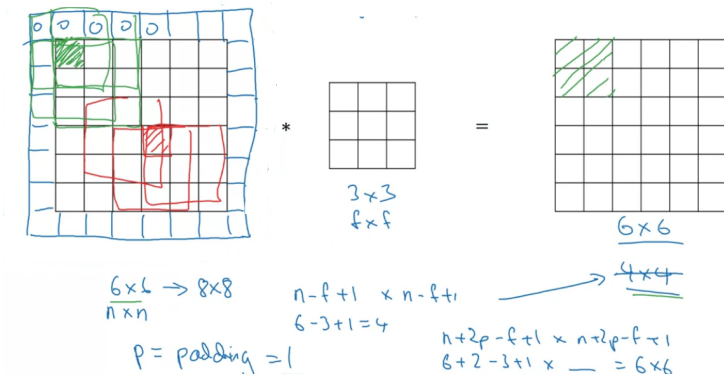
- Resolvido pelo **padding**: adição de uma fileira de pixels em volta



- Note que temos uma nova fórmula para as dimensões da saída
- Por convenção, preenche-se com zeros
- Podemos ter mais fileiras em volta (ex. $p = 2$)

Padding

- Resolvido pelo **padding**: adição de uma fileira de pixels em volta



- Note que temos uma nova fórmula para as dimensões da saída
- Por convenção, preenche-se com zeros
- Podemos ter mais fileiras em volta (ex. $p = 2$)

Padding

- Duas escolhas comuns para p : *valid* e *same convolution*

→ no padding

“Valid”: $n \times n$ $\ast$ $f \times f$ $\rightarrow \frac{n-f+1}{1} \times \frac{n-f+1}{1}$
 6×6 $\ast$ 3×3 $\rightarrow 4 \times 4$

“Same”: Pad so that output size is the same as the input size.

$$n+2p-f+1 \times n+2p-f+1$$

$$\cancel{n+2p-f+1} = \cancel{n} \Rightarrow \boxed{p = \frac{f-1}{2}}$$

$$3 \times 3 \quad p = \frac{3-1}{2} = 1 \quad \left| \begin{array}{l} 5 \times 5 \\ f=5 \end{array} \right. \quad p=2$$

Padding

- CURIOSIDADE: Os filtros costumam ter sempre dimensão ímpar (3x3, 5x5, 7x7) - razões:
 - No filtro par, o *padding* na convolução válida teria que ser assimétrico;
 - O filtro ímpar tem um pixel central, o que é muitas vezes conveniente em visão computacional

Padding

- CURIOSIDADE: Os filtros costumam ter sempre dimensão ímpar (3x3, 5x5, 7x7) - razões:
 - No filtro par, o *padding* na convolução válida teria que ser assimétrico;
 - O filtro ímpar tem um pixel central, o que é muitas vezes conveniente em visão computacional

Padding

- CURIOSIDADE: Os filtros costumam ter sempre dimensão ímpar (3x3, 5x5, 7x7) - razões:
 - No filtro par, o *padding* na convolução válida teria que ser assimétrico;
 - O filtro ímpar tem um pixel central, o que é muitas vezes conveniente em visão computacional

Strided Convolution

Na convolução com *stride* s , o bloco azul vai se deslocar de s em s

2 ³	3 ⁴	7 ⁴	4	6	2	9
6 ¹	6 ⁰	9 ²	8	7	4	3
3 ⁻¹	4 ⁰	8 ³	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7x7

*

3	4	4
1	0	2
-1	0	3

3x3
stride = 2

=

91.		

Strided Convolution

Na convolução com *stride* s , o bloco azul vai se deslocar de s em s

2 ³	3 ⁴	7 ⁴	4	6	2	9
6 ¹	6 ⁰	9 ²	8	7	4	3
3 ⁻¹	4 ⁰	8 ³	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7x7

3	4	4
1	0	2
-1	0	3

3x3

stride = 2

*

91.		

=

2	3	7 ³	4 ⁴	6 ⁴	2	9
6	6	9 ¹	8 ⁰	7 ²	4	3
3	4	8 ⁻¹	3 ⁰	8 ³	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7x7

3	4	4
1	0	2
-1	0	3

3x3

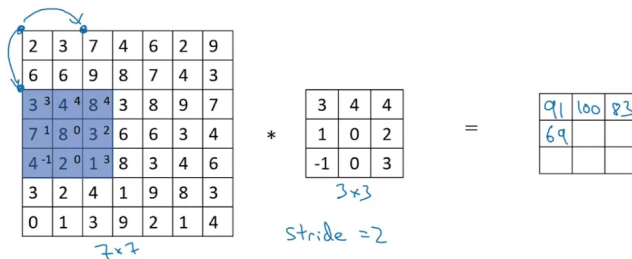
stride = 2

*

91	100	

=

Strided Convolution



Strided Convolution

Diagram illustrating a 2D Strided Convolution operation:

Input Matrix (7x7):

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3 ³	4 ⁴	6 ⁴
3	2	4	1	9 ¹	8 ⁰	3 ²
0	1	3	9	2 ⁻¹	1 ⁰	4 ³

Kernel Matrix (3x3):

3	4	4
1	0	2
-1	0	3

Output Matrix (3x3):

91	100	83
69	01	127
44	72	74

Handwritten notes:

- $n \times n$ * $f \times f$
- padding p
- stride s
- $s = 2$
- $\text{stride} = 2$
- $\lfloor z \rfloor = \text{floor}(z)$
- Formula for output size: $\left\lfloor \frac{n+p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+p-f}{s} + 1 \right\rfloor$
- Calculation: $\frac{7+0-3}{2} + 1 = \frac{4}{2} + 1 = 3$

Note que temos uma nova fórmula para a dimensão da saída, incluindo s

NOTA: Usa-se *floor* porque, por convenção, o filtro deve ficar inteiro sobre a imagem

Strided Convolution

Diagram illustrating a Strided Convolution operation:

Input (7x7):

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3 ³	4 ⁴	6 ⁴
3	2	4	1	9 ¹	8 ⁰	3 ²
0	1	3	9	2 ⁻¹	1 ⁰	4 ³

Kernel (3x3):

3	4	4
1	0	2
-1	0	3

Output (3x3):

91	100	83
69	01	127
44	72	74

Handwritten notes:

- $n \times n$ * $f \times f$
- padding p stride s
- $s = 2$
- $\lfloor \frac{n+p-f}{s} + 1 \rfloor \times \lfloor \frac{n+p-f}{s} + 1 \rfloor$
- $\frac{7+0-3}{2} + 1 = \frac{4}{2} + 1 = 3$
- $\lfloor z \rfloor = \text{floor}(z)$

Note que temos uma nova fórmula para a dimensão da saída, incluindo s

NOTA: Usa-se *floor* porque, por convenção, o filtro deve ficar inteiro sobre a imagem

Strided Convolution

Diagram illustrating a Strided Convolution operation:

Input (7x7):

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3 ³	4 ⁴	6 ⁴
3	2	4	1	9 ¹	8 ⁰	3 ²
0	1	3	9	2 ⁻¹	1 ⁰	4 ³

Kernel (3x3):

3	4	4
1	0	2
-1	0	3

Stride = 2

Output (3x3):

91	100	83
69	01	127
44	72	74

Formula for output dimensions:

$$\left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$$

Calculation:

$$\frac{7 + 0 - 3}{2} + 1 = \frac{4}{2} + 1 = 3$$

Note que temos uma nova fórmula para a dimensão da saída, incluindo s

NOTA: Usa-se *floor* porque, por convenção, o filtro deve ficar inteiro sobre a imagem

Detalhe Técnico

- Em livros de matemática e processamento de sinais, o que se chama de convolução é feito com o filtro espelhado, na horizontal e vertical
- Isso dá associatividade à operação, o que é útil em processamento de sinais, mas não em *deep learning*
- Por convenção aqui, a operação sem o espelhamento é chamada de convolução
- No processamento de sinais, essa operação sem espelhamento é chamada de *cross-correlation*

Detalhe Técnico

- Em livros de matemática e processamento de sinais, o que se chama de convolução é feito com o filtro espelhado, na horizontal e vertical
- Isso dá associatividade à operação, o que é útil em processamento de sinais, mas não em *deep learning*
- Por convenção aqui, a operação sem o espelhamento é chamada de convolução
- No processamento de sinais, essa operação sem espelhamento é chamada de *cross-correlation*

Detalhe Técnico

- Em livros de matemática e processamento de sinais, o que se chama de convolução é feito com o filtro espelhado, na horizontal e vertical
- Isso dá associatividade à operação, o que é útil em processamento de sinais, mas não em *deep learning*
- Por convenção aqui, a operação sem o espelhamento é chamada de convolução
- No processamento de sinais, essa operação sem espelhamento é chamada de *cross-correlation*

Detalhe Técnico

- Em livros de matemática e processamento de sinais, o que se chama de convolução é feito com o filtro espelhado, na horizontal e vertical
- Isso dá associatividade à operação, o que é útil em processamento de sinais, mas não em *deep learning*
- Por convenção aqui, a operação sem o espelhamento é chamada de convolução
- No processamento de sinais, essa operação sem espelhamento é chamada de *cross-correlation*

Outline

- 1 Visão Computacional
- 2 Convolução
- 3 Outras Operações Importantes
- 4 Convoluções sobre Volumes**
- 5 Uma Camada da Rede Convolutacional
- 6 Exemplo de uma Rede Convolutacional Simples
- 7 Camadas de *Pooling*
- 8 Exemplo Mais Completo de CNN
- 9 Por que Convoluções?

Convolução sobre um Volume 3D

- E a detecção de bordas em uma imagem RGB colorida $6 \times 6 \times 3$?
- O filtro agora é 3D também, no caso, $3 \times 3 \times 3$
- Essa dimensão a mais é o **número de canais** e precisa ser igual na imagem de entrada e no filtro
- Saída continua 4×4

Convolução sobre um Volume 3D

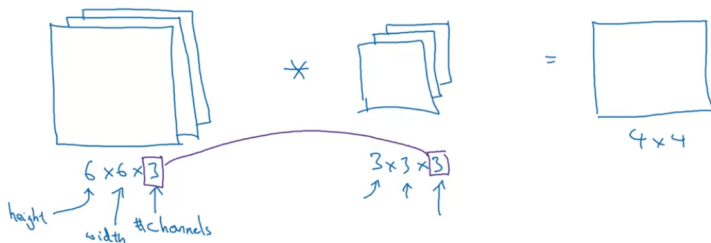
- E a detecção de bordas em uma imagem RGB colorida $6 \times 6 \times 3$?
- O filtro agora é 3D também, no caso, $3 \times 3 \times 3$
- Essa dimensão a mais é o **número de canais** e precisa ser igual na imagem de entrada e no filtro
- Saída continua 4×4

Convolução sobre um Volume 3D

- E a detecção de bordas em uma imagem RGB colorida $6 \times 6 \times 3$?
- O filtro agora é 3D também, no caso, $3 \times 3 \times 3$
- Essa dimensão a mais é o **número de canais** e precisa ser igual na imagem de entrada e no filtro
- Saída continua 4×4

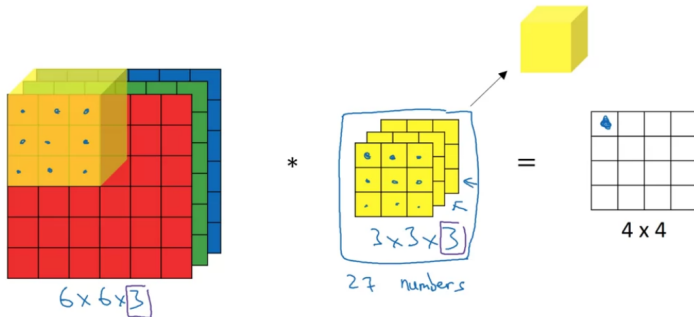
Convolução sobre um Volume 3D

- E a detecção de bordas em uma imagem RGB colorida $6 \times 6 \times 3$?
- O filtro agora é 3D também, no caso, $3 \times 3 \times 3$
- Essa dimensão a mais é o **número de canais** e precisa ser igual na imagem de entrada e no filtro
- Saída continua 4×4



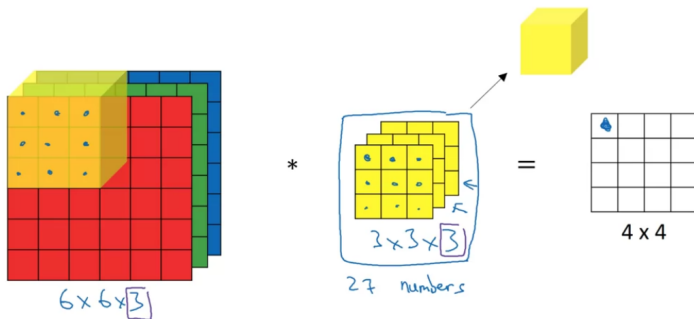
Convolução sobre um Volume 3D

- Primeiro posicionamos o cubo do filtro no canto inferior esquerdo da imagem 3D
- Então os valores são, como antes, multiplicados ponto a ponto nos 3 canais e somados
- Porém agora são 27 valores multiplicados



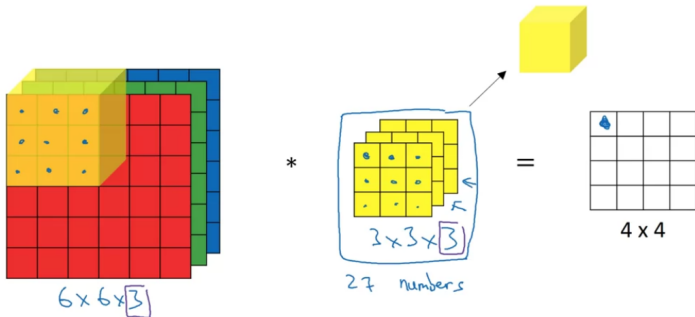
Convolução sobre um Volume 3D

- Primeiro posicionamos o cubo do filtro no canto inferior esquerdo da imagem 3D
- Então os valores são, como antes, multiplicados ponto a ponto nos 3 canais e somados
- Porém agora são 27 valores multiplicados



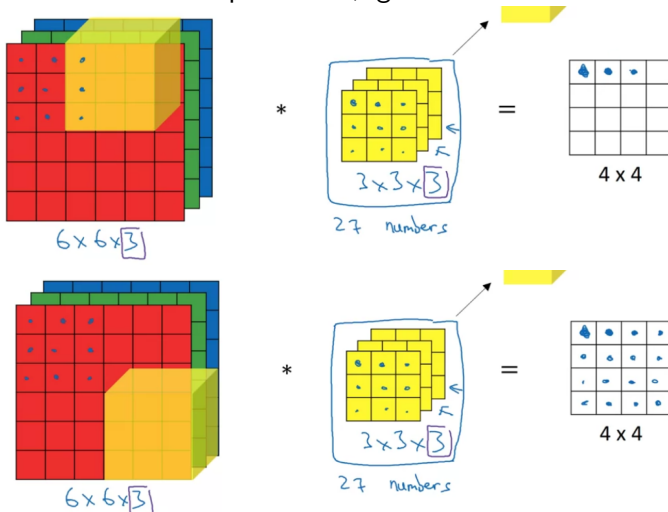
Convolução sobre um Volume 3D

- Primeiro posicionamos o cubo do filtro no canto inferior esquerdo da imagem 3D
- Então os valores são, como antes, multiplicados ponto a ponto nos 3 canais e somados
- Porém agora são 27 valores multiplicados



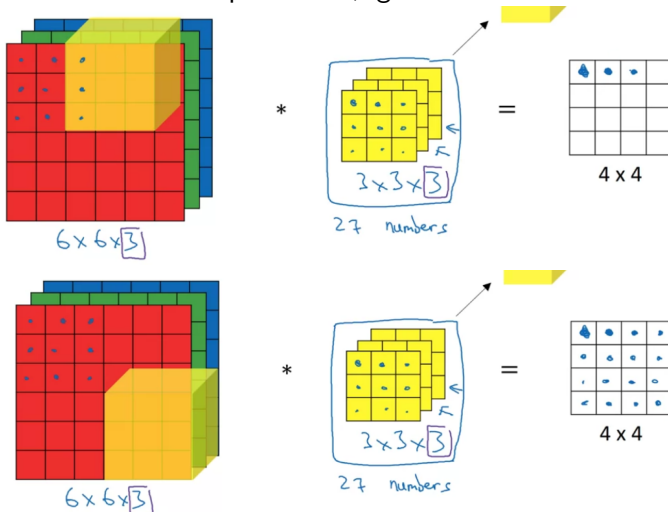
Convolução sobre um Volume 3D

- No próximo valor, deslocamos o cubo uma posição para a esquerda, depois para baixo e assim por diante, igual ao 2D



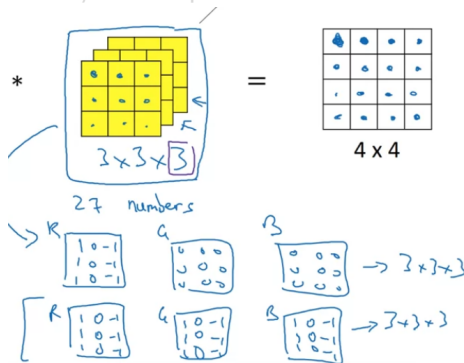
Convolução sobre um Volume 3D

- No próximo valor, deslocamos o cubo uma posição para a esquerda, depois para baixo e assim por diante, igual ao 2D



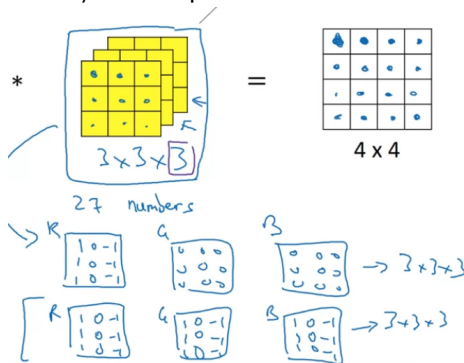
Convolução sobre um Volume 3D

- Permite, por exemplo, detectar bordas em apenas uma cor (ex. vermelho), usando os valores já vistos no 2D apenas no canal R e 0 nos demais
- Ou detectar em todas as cores replicando o filtro 2D nos 3 canais e assim muitas combinações são possíveis



Convolução sobre um Volume 3D

- Permite, por exemplo, detectar bordas em apenas uma cor (ex. vermelho), usando os valores já vistos no 2D apenas no canal R e 0 nos demais
- Ou detectar em todas as cores replicando o filtro 2D nos 3 canais e assim muitas combinações são possíveis



Convolução sobre um Volume 3D

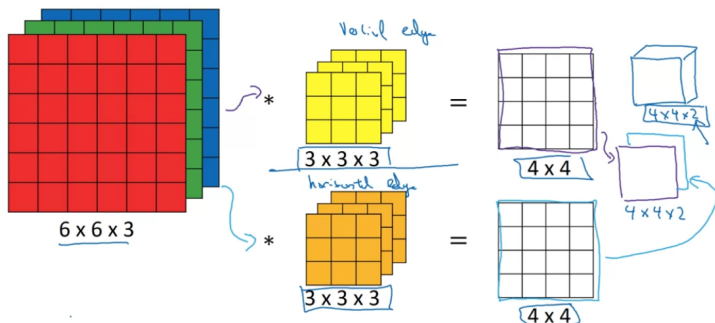
- E para detectar bordas verticais, horizontais, em 45° , etc. ao mesmo tempo? Ou seja, múltiplos filtros?
- Para 2 filtros (p.ex., borda vertical e horizontal), temos 2 saídas 4×4
- Empilhamos essas saídas, gerando uma saída 3D $4 \times 4 \times 2$

Convolução sobre um Volume 3D

- E para detectar bordas verticais, horizontais, em 45° , etc. ao mesmo tempo? Ou seja, múltiplos filtros?
- Para 2 filtros (p.ex., borda vertical e horizontal), temos 2 saídas 4×4
- Empilhamos essas saídas, gerando uma saída 3D $4 \times 4 \times 2$

Convolução sobre um Volume 3D

- E para detectar bordas verticais, horizontais, em 45° , etc. ao mesmo tempo? Ou seja, múltiplos filtros?
- Para 2 filtros (p.ex., borda vertical e horizontal), temos 2 saídas 4×4
- Empilhamos essas saídas, gerando uma saída 3D $4 \times 4 \times 2$



Convolução sobre um Volume 3D

- Dimensões:

Summary: $n \times n \times n_c$ $\times$ $f \times f \times n_c$ $\rightarrow$ $\frac{n-f+1}{4} \times \frac{n-f+1}{4} \times n'_c$

$6 \times 6 \times 3$ $3 \times 3 \times 3$ $4 \times 4 \times 2$ $\uparrow$ #filters

- Essa é a fórmula sem *stride* e *padding*; se tivesse, seria adaptada como já vimos
- Mais importante do que operar em imagens RGB, a convolução em volumes permite que se detectem vários *features* ao mesmo tempo
- NOTA: O número de canais é também chamado de **profundidade** do volume 3D, mas isso pode gerar confusão com o número de camadas

Convolução sobre um Volume 3D

- Dimensões:

Summary: $n \times n \times n_c$ $\times$ $f \times f \times n_c$ $\rightarrow$ $\frac{n-f+1}{4} \times \frac{n-f+1}{4} \times n'_c$

$6 \times 6 \times 3$ $3 \times 3 \times 3$ $4 \times 4 \times 2$ $\uparrow$ #filters

- Essa é a fórmula sem *stride* e *padding*; se tivesse, seria adaptada como já vimos
- Mais importante do que operar em imagens RGB, a convolução em volumes permite que se detectem vários *features* ao mesmo tempo
- NOTA: O número de canais é também chamado de **profundidade** do volume 3D, mas isso pode gerar confusão com o número de camadas

Convolução sobre um Volume 3D

- Dimensões:

Summary: $n \times n \times n_c$ $\times$ $f \times f \times n_c$ $\rightarrow$ $\frac{n-f+1}{4} \times \frac{n-f+1}{4} \times n'_c$

$6 \times 6 \times 3$ $3 \times 3 \times 3$ $4 \times 4 \times 2$ $\uparrow$ #filters

- Essa é a fórmula sem *stride* e *padding*; se tivesse, seria adaptada como já vimos
- Mais importante do que operar em imagens RGB, a convolução em volumes permite que se detectem vários *features* ao mesmo tempo
- NOTA: O número de canais é também chamado de **profundidade** do volume 3D, mas isso pode gerar confusão com o número de camadas

Convolução sobre um Volume 3D

- Dimensões:

Summary: $n \times n \times n_c$ $\times$ $f \times f \times n_c$ $\rightarrow$ $\frac{n-f+1}{4} \times \frac{n-f+1}{4} \times n'_c$

$6 \times 6 \times 3$ $3 \times 3 \times 3$ $4 \times 4 \times 2$ $\uparrow$ #filters

- Essa é a fórmula sem *stride* e *padding*; se tivesse, seria adaptada como já vimos
- Mais importante do que operar em imagens RGB, a convolução em volumes permite que se detectem vários *features* ao mesmo tempo
- NOTA: O número de canais é também chamado de **profundidade** do volume 3D, mas isso pode gerar confusão com o número de camadas

Outline

- 1 Visão Computacional
- 2 Convolução
- 3 Outras Operações Importantes
- 4 Convoluções sobre Volumes
- 5 Uma Camada da Rede Convolutacional**
- 6 Exemplo de uma Rede Convolutacional Simples
- 7 Camadas de *Pooling*
- 8 Exemplo Mais Completo de CNN
- 9 Por que Convoluções?

Camada Convolutacional

- Para formar uma camada da rede convolutacional, somamos a saída 4×4 com um *bias*, que é um número real somado a toda a matriz 4×4 por *broadcasting*
- Cada matriz 4×4 vai ter um *bias* diferente
- Em seguida, aplica-se uma não-linearidade ReLU
- Por fim, as saídas de cada filtro são empilhadas gerando uma saída $4 \times 4 \times 2$

Camada Convolutacional

- Para formar uma camada da rede convolutacional, somamos a saída 4×4 com um *bias*, que é um número real somado a toda a matriz 4×4 por *broadcasting*
- Cada matriz 4×4 vai ter um *bias* diferente
- Em seguida, aplica-se uma não-linearidade ReLU
- Por fim, as saídas de cada filtro são empilhadas gerando uma saída $4 \times 4 \times 2$

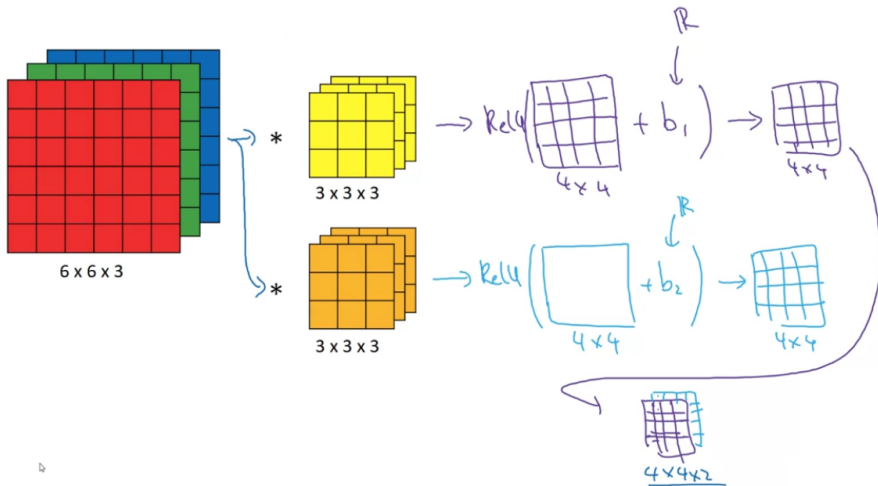
Camada Convolutacional

- Para formar uma camada da rede convolutacional, somamos a saída 4×4 com um *bias*, que é um número real somado a toda a matriz 4×4 por *broadcasting*
- Cada matriz 4×4 vai ter um *bias* diferente
- Em seguida, aplica-se uma não-linearidade ReLU
- Por fim, as saídas de cada filtro são empilhadas gerando uma saída $4 \times 4 \times 2$

Camada Convolutacional

- Para formar uma camada da rede convolutacional, somamos a saída 4×4 com um *bias*, que é um número real somado a toda a matriz 4×4 por *broadcasting*
- Cada matriz 4×4 vai ter um *bias* diferente
- Em seguida, aplica-se uma não-linearidade ReLU
- Por fim, as saídas de cada filtro são empilhadas gerando uma saída $4 \times 4 \times 2$

Camada Convolucional



Camada Convolutacional vs. Rede Clássica

- Lembrando do *forward* da rede clássica:

$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

- Agora, a imagem de entrada é $a^{[0]}$ e os múltiplos filtros são como $W^{[1]}$
- Convolução entra no lugar de $W^{[1]}a^{[0]}$
- Convolução mais *bias* está no lugar de $z^{[1]}$

Camada Convolutacional vs. Rede Clássica

- Lembrando do *forward* da rede clássica:

$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

- Agora, a imagem de entrada é $a^{[0]}$ e os múltiplos filtros são como $W^{[1]}$
- Convolução entra no lugar de $W^{[1]}a^{[0]}$
- Convolução mais *bias* está no lugar de $z^{[1]}$

Camada Convolutacional vs. Rede Clássica

- Lembrando do *forward* da rede clássica:

$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

- Agora, a imagem de entrada é $a^{[0]}$ e os múltiplos filtros são como $W^{[1]}$
- Convolução entra no lugar de $W^{[1]}a^{[0]}$
- Convolução mais *bias* está no lugar de $z^{[1]}$

Camada Convolutacional vs. Rede Clássica

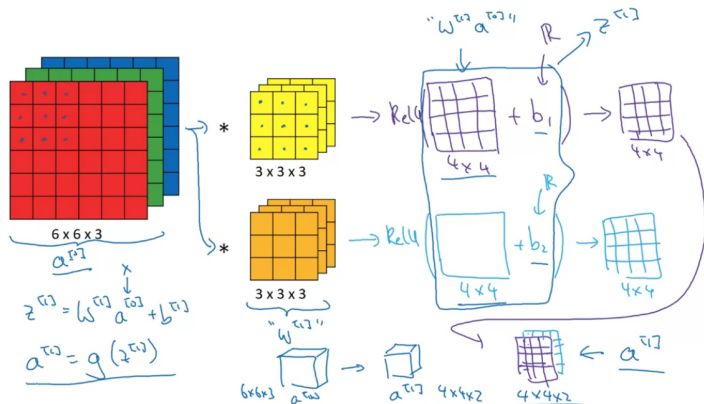
- Lembrando do *forward* da rede clássica:

$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

- Agora, a imagem de entrada é $a^{[0]}$ e os múltiplos filtros são como $W^{[1]}$
- Convolução entra no lugar de $W^{[1]}a^{[0]}$
- Convolução mais *bias* está no lugar de $z^{[1]}$

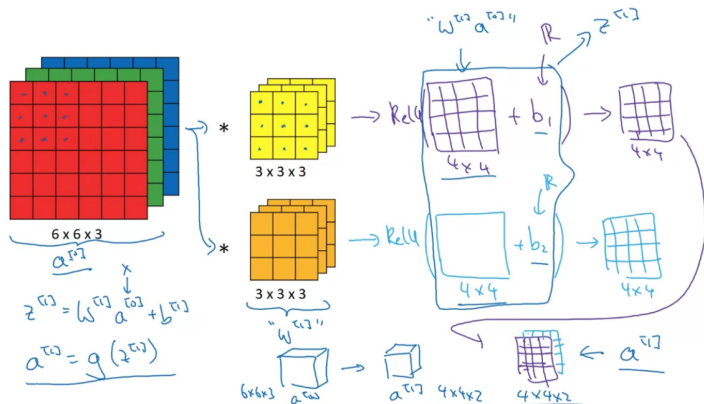
Camada Convolutiva vs. Rede Clássica



IMPORTANTE: No exemplo acima temos 2 filtros, mas poderíamos ter um número qualquer

Por exemplo, com 10 filtros, a saída seria $4 \times 4 \times 10$

Camada Convolucional vs. Rede Clássica

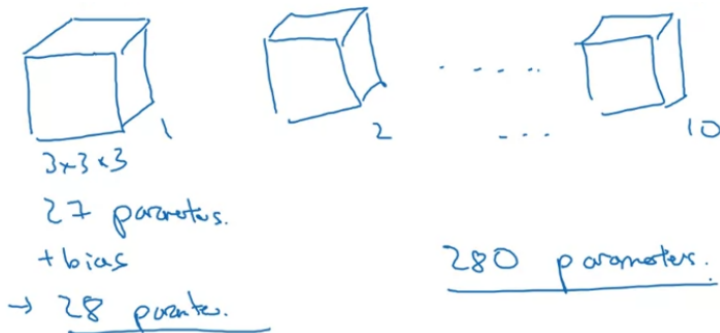


IMPORTANTE: No exemplo acima temos 2 filtros, mas poderíamos ter um número qualquer

Por exemplo, com 10 filtros, a saída seria $4 \times 4 \times 10$

Número de Parâmetros

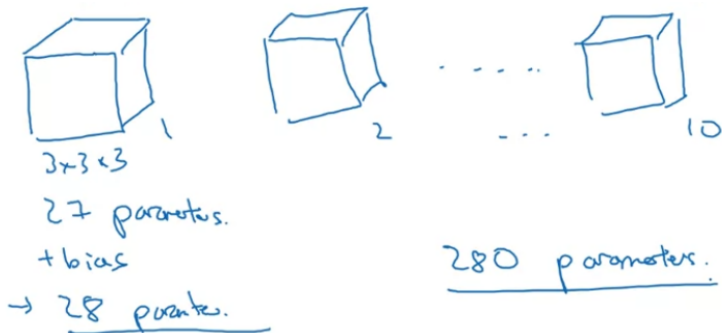
- Quantos parâmetros temos em uma camada com 10 filtros $3 \times 3 \times 3$?



- Uma boa coisa aqui é que não importa o tamanho da imagem (poderia ser 1000×1000); o número de parâmetros continuaria 280
- Isso torna as redes convolucionais menos propensas a *overfitting*

Número de Parâmetros

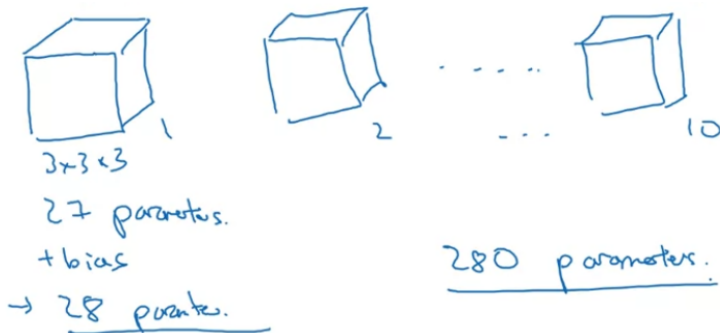
- Quantos parâmetros temos em uma camada com 10 filtros $3 \times 3 \times 3$?



- Uma boa coisa aqui é que não importa o tamanho da imagem (poderia ser 1000×1000); o número de parâmetros continuaria 280
- Isso torna as redes convolucionais menos propensas a *overfitting*

Número de Parâmetros

- Quantos parâmetros temos em uma camada com 10 filtros $3 \times 3 \times 3$?



- Uma boa coisa aqui é que não importa o tamanho da imagem (poderia ser 1000×1000); o número de parâmetros continuaria 280
- Isso torna as redes convolucionais menos propensas a *overfitting*

Notação e Dimensões na Camada /

$f^{[l]}$ = filter size

$p^{[l]}$ = padding

$s^{[l]}$ = stride

$n_c^{[l]}$ = number of filters

Each filter is: $f^{[l]} \times f^{[l]} \times n_c^{[l]}$

Activations: $a^{[l]} \rightarrow n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

Weights: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]} \times n_c^{[l]}$

bias: $n_c^{[l]} = (1, 1, 1, n_c^{[l]})$ ← #f: (ftrs in layer l).

Input: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$

Output: $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

- NOTA 1: Veremos mais à frente porque representamos o tensor do *bias* como $(1, 1, 1, n_c[l])$
- NOTA 2: O *padding* pode ser especificado pelo tipo de convolução: *same* ou *valid*

Notação e Dimensões na Camada /

$f^{[l]}$ = filter size

$p^{[l]}$ = padding

$s^{[l]}$ = stride

$n_c^{[l]}$ = number of filters

Each filter is: $f^{[l]} \times f^{[l]} \times n_c^{[l]}$

Activations: $a^{[l]} \rightarrow n_H \times n_W \times n_c^{[l]}$

Weights: $f^{[l]} \times f^{[l]} \times n_c^{[l]} \times n_c^{[l]}$

bias: $n_c^{[l]} = (1, 1, 1, n_c^{[l]})$ ← #f: (times in layer l.)

Input: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$

Output: $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

- NOTA 1: Veremos mais à frente porque representamos o tensor do *bias* como $(1, 1, 1, n_c[l])$
- NOTA 2: O *padding* pode ser especificado pelo tipo de convolução: *same* ou *valid*

Notação e Dimensões na Camada /

$f^{[l]}$ = filter size

$p^{[l]}$ = padding

$s^{[l]}$ = stride

$n_c^{[l]}$ = number of filters

Each filter is: $f^{[l]} \times f^{[l]} \times n_c^{[l]}$

Activations: $a^{[l]} \rightarrow n_H \times n_W \times n_c^{[l]}$

Weights: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]} \times n_c^{[l]}$

bias: $n_c^{[l]} = (1, 1, 1, n_c^{[l]})$ ← #f: (ftrs in layer l.)

Input: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$

Output: $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

- NOTA 1: Veremos mais à frente porque representamos o tensor do *bias* como $(1, 1, 1, n_c[l])$
- NOTA 2: O *padding* pode ser especificado pelo tipo de convolução: *same* ou *valid*

Notação e Dimensões na Camada /

- Valores de n_H e n_W :

$$n_{H/W}^{[l]} = \left\lfloor \frac{n_{H/W}^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

- Para m exemplos, no gradiente descendente em batch teríamos:

$$A^{[l]} \rightarrow m \times n_H^{[l]} \times n_W^{[l]} \times n_C^{[l]}$$

- NOTA: No código, é bem comum também o uso de n_C na frente, de modo que

$$A^{[l]} \rightarrow m \times n_C^{[l]} \times n_H^{[l]} \times n_W^{[l]}$$

Notação e Dimensões na Camada /

- Valores de n_H e n_W :

$$n_{H/W}^{[l]} = \left\lfloor \frac{n_{H/W}^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

- Para m exemplos, no gradiente descendente em batch teríamos:

$$A^{[l]} \rightarrow m \times n_H^{[l]} \times n_W^{[l]} \times n_C^{[l]}$$

- NOTA: No código, é bem comum também o uso de n_C na frente, de modo que

$$A^{[l]} \rightarrow m \times n_C^{[l]} \times n_H^{[l]} \times n_W^{[l]}$$

Notação e Dimensões na Camada /

- Valores de n_H e n_W :

$$n_{H/W}^{[l]} = \left\lfloor \frac{n_{H/W}^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

- Para m exemplos, no gradiente descendente em batch teríamos:

$$A^{[l]} \rightarrow m \times n_H^{[l]} \times n_W^{[l]} \times n_C^{[l]}$$

- NOTA: No código, é bem comum também o uso de n_C na frente, de modo que

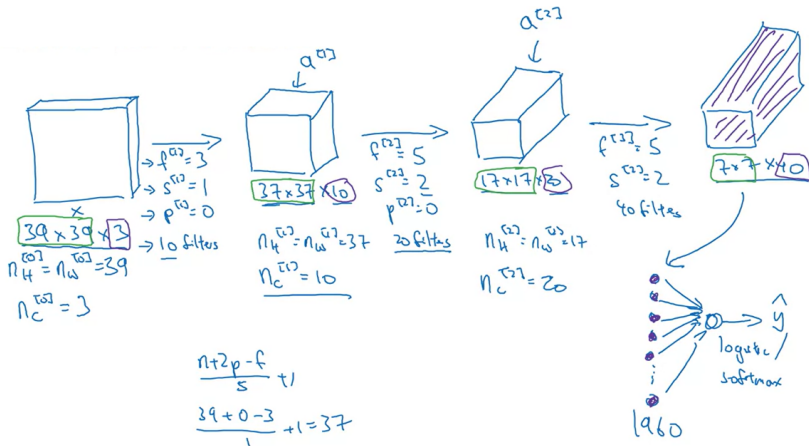
$$A^{[l]} \rightarrow m \times n_C^{[l]} \times n_H^{[l]} \times n_W^{[l]}$$

Outline

- 1 Visão Computacional
- 2 Convolução
- 3 Outras Operações Importantes
- 4 Convoluções sobre Volumes
- 5 Uma Camada da Rede Convolutacional
- 6 Exemplo de uma Rede Convolutacional Simples**
- 7 Camadas de *Pooling*
- 8 Exemplo Mais Completo de CNN
- 9 Por que Convoluções?

Exemplo

- Classificação de uma imagem 39x39 (x3)



Exemplo

- 10 filtros 3x3 na 1a camada convolucional
- Na 2a camada, temos 20 filtros 5x5 com *stride* 2
- Última camada convolucional com 40 filtros 7x7 e *stride* 2
- Volume final 7x7x40
- O mais usual é “achatar” (*flatten*) este volume para um vetor de 1960 componentes
- Este é entrada para uma regressão logística ou *softmax* dependendo do número de classes

Exemplo

- 10 filtros 3x3 na 1a camada convolutacional
- Na 2a camada, temos 20 filtros 5x5 com *stride* 2
- Última camada convolutacional com 40 filtros 7x7 e *stride* 2
- Volume final 7x7x40
- O mais usual é “achatar” (*flatten*) este volume para um vetor de 1960 componentes
- Este é entrada para uma regressão logística ou *softmax* dependendo do número de classes

Exemplo

- 10 filtros 3x3 na 1a camada convolucional
- Na 2a camada, temos 20 filtros 5x5 com *stride* 2
- Última camada convolucional com 40 filtros 7x7 e *stride* 2
- Volume final 7x7x40
- O mais usual é “achatar” (*flatten*) este volume para um vetor de 1960 componentes
- Este é entrada para uma regressão logística ou *softmax* dependendo do número de classes

Exemplo

- 10 filtros 3x3 na 1a camada convolucional
- Na 2a camada, temos 20 filtros 5x5 com *stride* 2
- Última camada convolucional com 40 filtros 7x7 e *stride* 2
- Volume final 7x7x40
- O mais usual é “achatar” (*flatten*) este volume para um vetor de 1960 componentes
- Este é entrada para uma regressão logística ou *softmax* dependendo do número de classes

Exemplo

- 10 filtros 3x3 na 1a camada convolucional
- Na 2a camada, temos 20 filtros 5x5 com *stride* 2
- Última camada convolucional com 40 filtros 7x7 e *stride* 2
- Volume final 7x7x40
- O mais usual é “achatar” (*flatten*) este volume para um vetor de 1960 componentes
- Este é entrada para uma regressão logística ou *softmax* dependendo do número de classes

Exemplo

- 10 filtros 3x3 na 1a camada convolucional
- Na 2a camada, temos 20 filtros 5x5 com *stride* 2
- Última camada convolucional com 40 filtros 7x7 e *stride* 2
- Volume final 7x7x40
- O mais usual é “achatar” (*flatten*) este volume para um vetor de 1960 componentes
- Este é entrada para uma regressão logística ou *softmax* dependendo do número de classes

Exemplo

- Tendência geral de a altura/largura irem diminuindo em cada camada e o número de canais aumentando
- Escolha de hiperparâmetros, como f , s e p é uma parte muito importante
- A rede convolutiva típica ainda tem outros dois tipos de camadas além da de **Convolução (CONV)**:
 - Pooling (POOL)
 - Fully connected (FC)

Exemplo

- Tendência geral de a altura/largura irem diminuindo em cada camada e o número de canais aumentando
- Escolha de hiperparâmetros, como f , s e p é uma parte muito importante
- A rede convolutiva típica ainda tem outros dois tipos de camadas além da de **Convolução (CONV)**:
 - Pooling (POOL)
 - Fully connected (FC)

Exemplo

- Tendência geral de a altura/largura irem diminuindo em cada camada e o número de canais aumentando
- Escolha de hiperparâmetros, como f , s e p é uma parte muito importante
- A rede convolutiva típica ainda tem outros dois tipos de camadas além da de **Convolução (CONV)**:
 - Pooling (POOL)
 - Fully connected (FC)

Exemplo

- Tendência geral de a altura/largura irem diminuindo em cada camada e o número de canais aumentando
- Escolha de hiperparâmetros, como f , s e p é uma parte muito importante
- A rede convolutiva típica ainda tem outros dois tipos de camadas além da de **Convolução (CONV)**:
 - **Pooling (POOL)**
 - Fully connected (FC)

Exemplo

- Tendência geral de a altura/largura irem diminuindo em cada camada e o número de canais aumentando
- Escolha de hiperparâmetros, como f , s e p é uma parte muito importante
- A rede convolutiva típica ainda tem outros dois tipos de camadas além da de **Convolução (CONV)**:
 - **Pooling (POOL)**
 - **Fully connected (FC)**

Outline

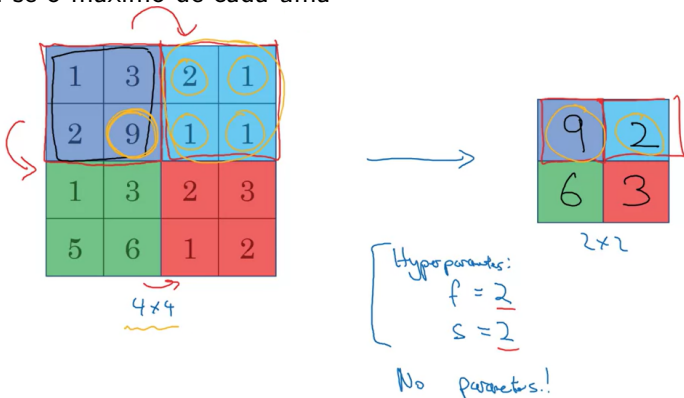
- 1 Visão Computacional
- 2 Convolução
- 3 Outras Operações Importantes
- 4 Convoluções sobre Volumes
- 5 Uma Camada da Rede Convolutacional
- 6 Exemplo de uma Rede Convolutacional Simples
- 7 Camadas de *Pooling***
- 8 Exemplo Mais Completo de CNN
- 9 Por que Convoluções?

Camadas de *Pooling*

- Reduzem o tamanho da representação para acelerar a computação e tornar alguns dos *features* detectados mais robustos
- *Max pooling* - uma imagem exemplo 4x4 é dividida em 4 regiões 2x2 e toma-se o máximo de cada uma

Camadas de Pooling

- Reduzem o tamanho da representação para acelerar a computação e tornar alguns dos *features* detectados mais robustos
- *Max pooling* - uma imagem exemplo 4x4 é dividida em 4 regiões 2x2 e toma-se o máximo de cada uma



Camadas de *Pooling*

- O tamanho do filtro f no caso é o tamanho da sub-região e temos também o *stride*
- INTUIÇÃO: Se um *feature* (ex. borda, olho de um gato, etc.) estiver presente em qualquer lugar do quadrante, atribuir um valor alto ali
- Mas raramente alguém pensa nisso quando usa *max pooling*; a principal razão é que empiricamente funciona muito bem
- NOTA: Temos hiperparâmetros (f e s), não há nenhum parâmetro a ser aprendido pelo gradiente descendente

Camadas de *Pooling*

- O tamanho do filtro f no caso é o tamanho da sub-região e temos também o *stride*
- INTUIÇÃO: Se um *feature* (ex. borda, olho de um gato, etc.) estiver presente em qualquer lugar do quadrante, atribuir um valor alto ali
- Mas raramente alguém pensa nisso quando usa *max pooling*; a principal razão é que empiricamente funciona muito bem
- NOTA: Temos hiperperâmetros (f e s), não há nenhum parâmetro a ser aprendido pelo gradiente descendente

Camadas de *Pooling*

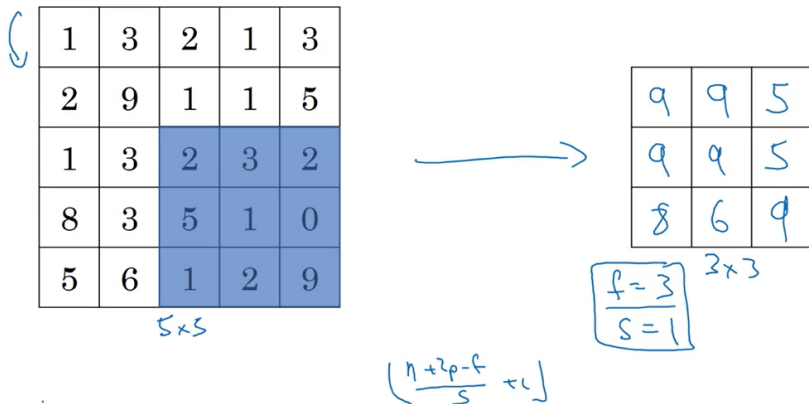
- O tamanho do filtro f no caso é o tamanho da sub-região e temos também o *stride*
- INTUIÇÃO: Se um *feature* (ex. borda, olho de um gato, etc.) estiver presente em qualquer lugar do quadrante, atribuir um valor alto ali
- Mas raramente alguém pensa nisso quando usa *max pooling*; a principal razão é que empiricamente funciona muito bem
- NOTA: Temos hiperparâmetros (f e s), não há nenhum parâmetro a ser aprendido pelo gradiente descendente

Camadas de *Pooling*

- O tamanho do filtro f no caso é o tamanho da sub-região e temos também o *stride*
- INTUIÇÃO: Se um *feature* (ex. borda, olho de um gato, etc.) estiver presente em qualquer lugar do quadrante, atribuir um valor alto ali
- Mas raramente alguém pensa nisso quando usa *max pooling*; a principal razão é que empiricamente funciona muito bem
- NOTA: Temos hiperparâmetros (f e s), não há nenhum parâmetro a ser aprendido pelo gradiente descendente

Pooling com Stride 1

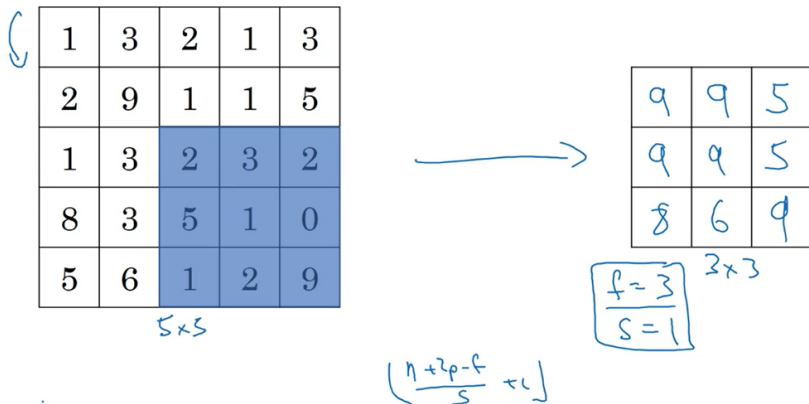
Exemplo com filtro 3x3 e imagem 5x5:



NOTA: A fórmula para o tamanho da saída continua funcionando!

Pooling com Stride 1

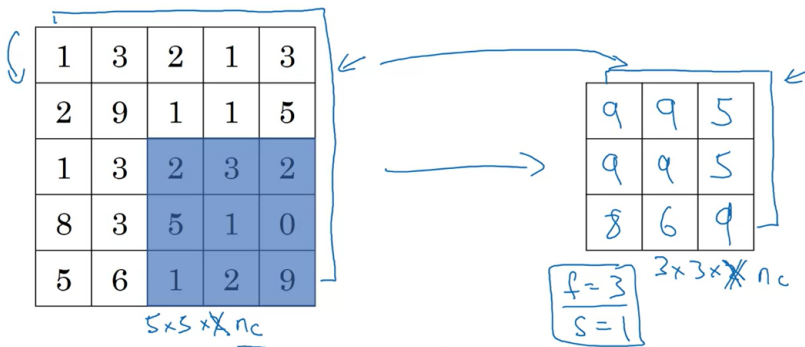
Exemplo com filtro 3x3 e imagem 5x5:



NOTA: A fórmula para o tamanho da saída continua funcionando!

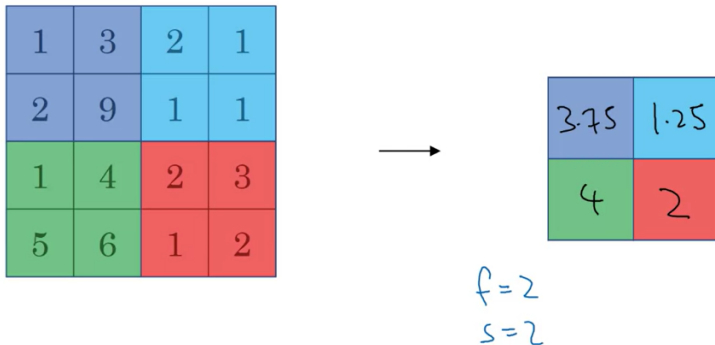
Pooling sobre um Volume 3D

- Saída tem o mesmo número de canais da entrada
- *Max pooling* independente em cada canal



Average Pooling

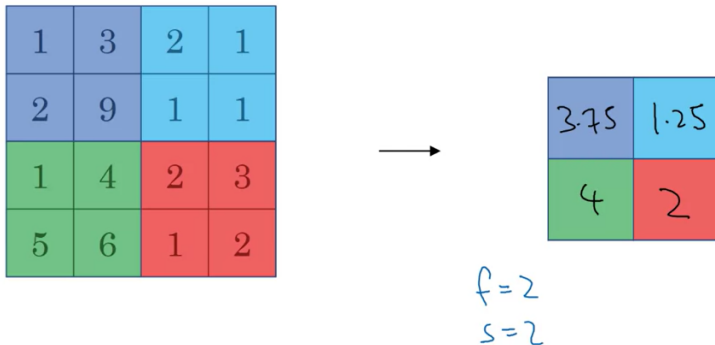
- Similar ao *max pooling*, mas em vez do valor máximo, calcula-se a média



- Max pooling* é muito mais comum atualmente
- Uma exceção de uso do *average pooling* é para colapsar um tensor, p.ex., de $7 \times 7 \times 1000$ para $1 \times 1 \times 1000$

Average Pooling

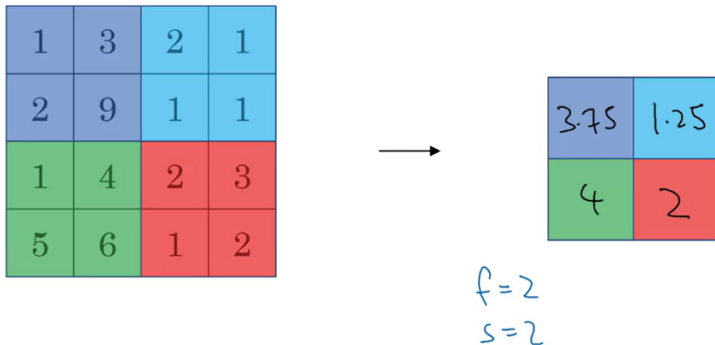
- Similar ao *max pooling*, mas em vez do valor máximo, calcula-se a média



- Max pooling* é muito mais comum atualmente
- Uma exceção de uso do *average pooling* é para colapsar um tensor, p.ex., de $7 \times 7 \times 1000$ para $1 \times 1 \times 1000$

Average Pooling

- Similar ao *max pooling*, mas em vez do valor máximo, calcula-se a média



- Max pooling* é muito mais comum atualmente
- Uma exceção de uso do *average pooling* é para colapsar um tensor, p.ex., de $7 \times 7 \times 1000$ para $1 \times 1 \times 1000$

Dimensões e Hiperparâmetros

- Muito raro se usar *padding* no *pooling*
- $f = 2, s = 2$ e $f = 3, s = 2$ são combinações de hiperparâmetros muito populares
- Resumo abaixo, incluindo a dimensão da saída:

Dimensões e Hiperparâmetros

- Muito raro se usar *padding* no *pooling*
- $f = 2, s = 2$ e $f = 3, s = 2$ são combinações de hiperparâmetros muito populares
- Resumo abaixo, incluindo a dimensão da saída:

Dimensões e Hiperparâmetros

- Muito raro se usar *padding* no *pooling*
- $f = 2, s = 2$ e $f = 3, s = 2$ são combinações de hiperparâmetros muito populares
- Resumo abaixo, incluindo a dimensão da saída:

Hyperparameters:

f : filter size
 s : stride
Max or average pooling

~~p : padding~~

No parameters to learn!

$$\begin{array}{c}
 n_H \times n_W \times \underline{n_C} \\
 \downarrow \\
 \left\lfloor \frac{n_H - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W - f}{s} + 1 \right\rfloor \\
 \times \underline{n_C}
 \end{array}$$

Outline

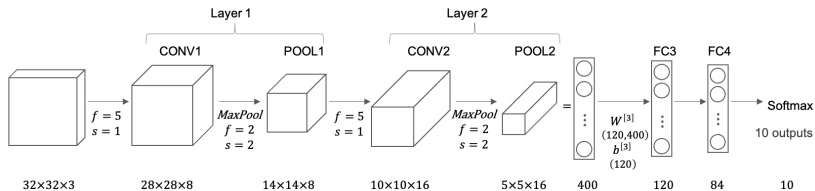
- 1 Visão Computacional
- 2 Convolução
- 3 Outras Operações Importantes
- 4 Convoluções sobre Volumes
- 5 Uma Camada da Rede Convolutacional
- 6 Exemplo de uma Rede Convolutacional Simples
- 7 Camadas de *Pooling*
- 8 Exemplo Mais Completo de CNN**
- 9 Por que Convoluções?

Exemplo de CNN

- Reconhecer dígito em uma imagem 32x32x3
- Modelo bem parecido com a clássica LeNet-5
- NOTA: Há divergências sobre se a camada convolucional e a de *pooling* subsequente formam camadas separadas ou uma única camada
- Quando se conta o número de camadas, é comum que se considere apenas as camadas com parâmetros a serem aprendidos

Exemplo de CNN

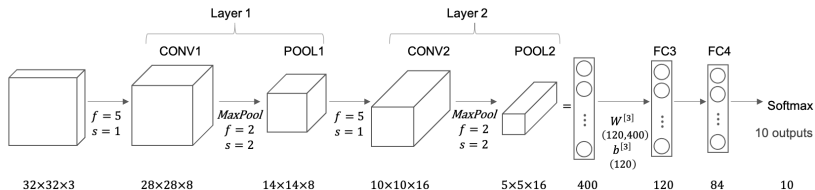
- Reconhecer dígito em uma imagem $32 \times 32 \times 3$
- Modelo bem parecido com a clássica LeNet-5



- NOTA: Há divergências sobre se a camada convolucional e a de *pooling* subsequente formam camadas separadas ou uma única camada
- Quando se conta o número de camadas, é comum que se considere apenas as camadas com parâmetros a serem aprendidos

Exemplo de CNN

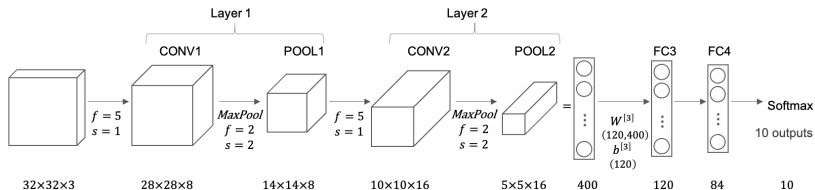
- Reconhecer dígito em uma imagem $32 \times 32 \times 3$
- Modelo bem parecido com a clássica LeNet-5



- NOTA: Há divergências sobre se a camada convolucional e a de *pooling* subsequente formam camadas separadas ou uma única camada
- Quando se conta o número de camadas, é comum que se considere apenas as camadas com parâmetros a serem aprendidos

Exemplo de CNN

- Reconhecer dígito em uma imagem $32 \times 32 \times 3$
- Modelo bem parecido com a clássica LeNet-5



- NOTA: Há divergências sobre se a camada convolucional e a de *pooling* subsequente formam camadas separadas ou uma única camada
- Quando se conta o número de camadas, é comum que se considere apenas as camadas com parâmetros a serem aprendidos

Exemplo de CNN

- Assim a camada convolucional + *pooling* é tratada como uma camada única.
- Na figura, temos CONV1 e POOL1 indicando que ambos são partes da Camada 1
- A saída de POOL2 é achatada para um vetor de 400 componentes usada como entrada de uma rede *fully-connected* (FC)
- Esta é a rede neural clássica que já vimos, que recebe este nome porque todas as unidades de camadas subsequentes se conectam entre si

Exemplo de CNN

- Assim a camada convolucional + *pooling* é tratada como uma camada única.
- Na figura, temos CONV1 e POOL1 indicando que ambos são partes da Camada 1
- A saída de POOL2 é achatada para um vetor de 400 componentes usada como entrada de uma rede *fully-connected* (FC)
- Esta é a rede neural clássica que já vimos, que recebe este nome porque todas as unidades de camadas subsequentes se conectam entre si

Exemplo de CNN

- Assim a camada convolucional + *pooling* é tratada como uma camada única.
- Na figura, temos CONV1 e POOL1 indicando que ambos são partes da Camada 1
- A saída de POOL2 é achatada para um vetor de 400 componentes usada como entrada de uma rede *fully-connected* (FC)
- Esta é a rede neural clássica que já vimos, que recebe este nome porque todas as unidades de camadas subsequentes se conectam entre si

Exemplo de CNN

- Assim a camada convolucional + *pooling* é tratada como uma camada única.
- Na figura, temos CONV1 e POOL1 indicando que ambos são partes da Camada 1
- A saída de POOL2 é achatada para um vetor de 400 componentes usada como entrada de uma rede *fully-connected* (FC)
- Esta é a rede neural clássica que já vimos, que recebe este nome porque todas as unidades de camadas subsequentes se conectam entre si

Hiperparâmetros

- Note que temos muitos hiperparâmetros para decidir
- É comum buscar na literatura por um conjunto fechado de hiperparâmetros que definem uma **arquitetura**, que sabidamente funciona bem em outros problemas
- Novamente, à medida que vamos mais profundo na rede, a altura e largura **diminuem** e o número de canais **aumenta**
- NOTA: O padrão CONV-POOL-CONV-POOL-FC-FC-FC-softmax é muito comum em redes convolucionais

Hiperparâmetros

- Note que temos muitos hiperparâmetros para decidir
- É comum buscar na literatura por um conjunto fechado de hiperparâmetros que definem uma **arquitetura**, que sabidamente funciona bem em outros problemas
- Novamente, à medida que vamos mais profundo na rede, a altura e largura **diminuem** e o número de canais **aumenta**
- NOTA: O padrão CONV-POOL-CONV-POOL-FC-FC-FC-softmax é muito comum em redes convolucionais

Hiperparâmetros

- Note que temos muitos hiperparâmetros para decidir
- É comum buscar na literatura por um conjunto fechado de hiperparâmetros que definem uma **arquitetura**, que sabidamente funciona bem em outros problemas
- Novamente, à medida que vamos mais profundo na rede, a altura e largura **diminuem** e o número de canais **aumenta**
- NOTA: O padrão CONV-POOL-CONV-POOL-FC-FC-FC-softmax é muito comum em redes convolucionais

Hiperparâmetros

- Note que temos muitos hiperparâmetros para decidir
- É comum buscar na literatura por um conjunto fechado de hiperparâmetros que definem uma **arquitetura**, que sabidamente funciona bem em outros problemas
- Novamente, à medida que vamos mais profundo na rede, a altura e largura **diminuem** e o número de canais **aumenta**
- NOTA: O padrão CONV-POOL-CONV-POOL-FC-FC-FC-softmax é muito comum em redes convolucionais

Número de Parâmetros

layer	activation shape	activation size	# parameters
Input	(32,32,3)	3072	0
CONV1 (f=5,s=1)	(28,28,8)	6272	608 $= (5*5*3+1)*8$
POOL1	(14,14,8)	1,568	0
CONV2 (f=5,s=1)	(10,10,16)	1600	3216 $= (5*5*8+1)*16$
POOL2	(5,5,16)	400	0
FC3	(120,1)	120	48120 $= 400*120+120$
FC4	(84,1)	84	10164 $= 120*84+84$
softmax	(10,1)	10	850 $= 84*10+10$

Número de Parâmetros

- Poucos parâmetros nas camadas convolucionais em relação às *fully-connected*
- Dimensão da ativação vai ficando gradualmente menor cada camada
- Se essa redução for muito rápida, a performance da rede piora
- A forma como esses blocos são combinados tomou muitos anos de pesquisa na visão computacional
- Ver exemplos concretos de redes nos dá *insights* sobre como isso é feito

Número de Parâmetros

- Poucos parâmetros nas camadas convolucionais em relação às *fully-connected*
- Dimensão da ativação vai ficando gradualmente menor cada camada
- Se essa redução for muito rápida, a performance da rede piora
- A forma como esses blocos são combinados tomou muitos anos de pesquisa na visão computacional
- Ver exemplos concretos de redes nos dá *insights* sobre como isso é feito

Número de Parâmetros

- Poucos parâmetros nas camadas convolucionais em relação às *fully-connected*
- Dimensão da ativação vai ficando gradualmente menor cada camada
- Se essa redução for muito rápida, a performance da rede piora
- A forma como esses blocos são combinados tomou muitos anos de pesquisa na visão computacional
- Ver exemplos concretos de redes nos dá *insights* sobre como isso é feito

Número de Parâmetros

- Poucos parâmetros nas camadas convolucionais em relação às *fully-connected*
- Dimensão da ativação vai ficando gradualmente menor cada camada
- Se essa redução for muito rápida, a performance da rede piora
- A forma como esses blocos são combinados tomou muitos anos de pesquisa na visão computacional
- Ver exemplos concretos de redes nos dá *insights* sobre como isso é feito

Número de Parâmetros

- Poucos parâmetros nas camadas convolucionais em relação às *fully-connected*
- Dimensão da ativação vai ficando gradualmente menor cada camada
- Se essa redução for muito rápida, a performance da rede piora
- A forma como esses blocos são combinados tomou muitos anos de pesquisa na visão computacional
- Ver exemplos concretos de redes nos dá *insights* sobre como isso é feito

Outline

- 1 Visão Computacional
- 2 Convolução
- 3 Outras Operações Importantes
- 4 Convoluções sobre Volumes
- 5 Uma Camada da Rede Convolucional
- 6 Exemplo de uma Rede Convolucional Simples
- 7 Camadas de *Pooling*
- 8 Exemplo Mais Completo de CNN
- 9 Por que Convoluções?

Vantagens das Camadas Convolucionais

- Compartilhamento de parâmetros e esparsidade de conexões
- Na CONV1 da figura acima, tínhamos 608 parâmetros
- Uma camada FC com os mesmos tamanhos de tensores teria $3072 \times 6272 \approx 19$ milhões de parâmetros!
- E isso para uma imagem bem pequena!

Vantagens das Camadas Convolucionais

- Compartilhamento de parâmetros e esparsidade de conexões
- Na CONV1 da figura acima, tínhamos 608 parâmetros
- Uma camada FC com os mesmos tamanhos de tensores teria $3072 \times 6272 \approx 19$ milhões de parâmetros!
- E isso para uma imagem bem pequena!

Vantagens das Camadas Convolucionais

- Compartilhamento de parâmetros e esparsidade de conexões
- Na CONV1 da figura acima, tínhamos 608 parâmetros
- Uma camada FC com os mesmos tamanhos de tensores teria $3072 \times 6272 \approx 19$ milhões de parâmetros!
- E isso para uma imagem bem pequena!

Vantagens das Camadas Convolucionais

- Compartilhamento de parâmetros e esparsidade de conexões
- Na CONV1 da figura acima, tínhamos 608 parâmetros
- Uma camada FC com os mesmos tamanhos de tensores teria $3072 \times 6272 \approx 19$ milhões de parâmetros!
- E isso para uma imagem bem pequena!

Vantagens das Camadas Convolucionais

- Razões para o número menor de parâmetros:
 - Compartilhamento de parâmetros - detector de *features* (seja de baixo nível como uma borda ou alto nível como o olho de um gato) é útil em qualquer posição da imagem
 - Esparsidade de conexões - cada saída depende (está conectada) a apenas $f \times f$ *features* de entradas
- NOTA: Outra propriedade bem conhecida das camadas convolucionais é sua **invariância a translação**: em todas as camadas o mesmo filtro passa por todas as posições

Vantagens das Camadas Convolucionais

- Razões para o número menor de parâmetros:
 - Compartilhamento de parâmetros - detector de *features* (seja de baixo nível como uma borda ou alto nível como o olho de um gato) é útil em qualquer posição da imagem
 - Esparsidade de conexões - cada saída depende (está conectada) a apenas $f \times f$ *features* de entradas
- NOTA: Outra propriedade bem conhecida das camadas convolucionais é sua **invariância a translação**: em todas as camadas o mesmo filtro passa por todas as posições

Vantagens das Camadas Convolucionais

- Razões para o número menor de parâmetros:
 - Compartilhamento de parâmetros - detector de *features* (seja de baixo nível como uma borda ou alto nível como o olho de um gato) é útil em qualquer posição da imagem
 - Esparsidade de conexões - cada saída depende (está conectada) a apenas $f \times f$ *features* de entradas
- NOTA: Outra propriedade bem conhecida das camadas convolucionais é sua **invariância a translação**: em todas as camadas o mesmo filtro passa por todas as posições

Vantagens das Camadas Convolucionais

- Razões para o número menor de parâmetros:
 - Compartilhamento de parâmetros - detector de *features* (seja de baixo nível como uma borda ou alto nível como o olho de um gato) é útil em qualquer posição da imagem
 - Esparsidade de conexões - cada saída depende (está conectada) a apenas $f \times f$ *features* de entradas
- NOTA: Outra propriedade bem conhecida das camadas convolucionais é sua **invariância a translação**: em todas as camadas o mesmo filtro passa por todas as posições

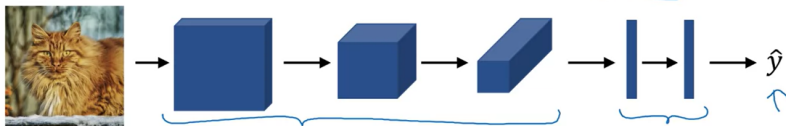
Juntando as Peças

- Treinamento idêntico ao que já vimos
- Definimos uma *loss*, inicializamos pesos e *biases* aleatórios e usamos um otimizador como o gradiente descendente, Adam, etc.

Juntando as Peças

- Treinamento idêntico ao que já vimos
- Definimos uma *loss*, inicializamos pesos e *biases* aleatórios e usamos um otimizador como o gradiente descendente, Adam, etc.

Training set $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$.



$$\text{Cost } J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$